KHULNA
UNIVERSITY OF
ENGINEERING AND
TECHNOLOGY

Akash Biswas
Lecturer, EEE
KUET

Department of Electrical and Electronic Engineering

Course Code : EE 4121

VLSI

Hardware Description Language

- **What is HDL?**
- HDL stands for Hardware Description Language. It is a programming language that is used to describe, simulate, and create hardware like digital circuits (ICS). HDL is mainly used to discover the faults in the design before implementing it in the hardware.

- In computer engineering, a hardware description language (HDL) is a specialized computer language used to describe the structure and behavior of electronic circuits, and most commonly, digital logic circuits

Hardware Description Languages (HDLs)

- VHDL and Verilog, early 1980s
- Both different from ordinary programming languages.
- VHDL – VHSIC HDL

  Very High Speed Integrated Circuit HDL
- Verilog – Verilog HDL
- VHDL syntax from ADA
- Verilog syntax from C (less intimidating)
- They enjoy 50/50 market share
- Design companies continue to use what they are used to

VHDL vs Verilog HDL

- Verilog allows faster simulation models through using the C language.
- VHDL has higher level design management features compared to Verilog.
- VHDL excellent for supporting of large systems.
- VHDL more strict in syntax.
- VHDL more popular in Europe and Australia while Verilog more in US and Asia.
- “Both languages are easy to learn and hard to master, and once you have learned one of these languages, you will have no trouble transitioning to the other” – VHDL Made Easy!

Difference between VHDL and Verilog

VHDL	Verilog
It allows the user to define data types.	It does not allow the user to define data types.
It supports the Multi-Dimensional array.	It does not support the Multi-Dimensional array.
It allows concurrent procedure calls.	It does not allow concurrent calls.
A mod operator is present.	A mod operator is not present.
Unary reduction operator is not present.	Unary reduction operator is present.
It is more difficult to learn.	It is easy to learn.

Introduction to VHDL

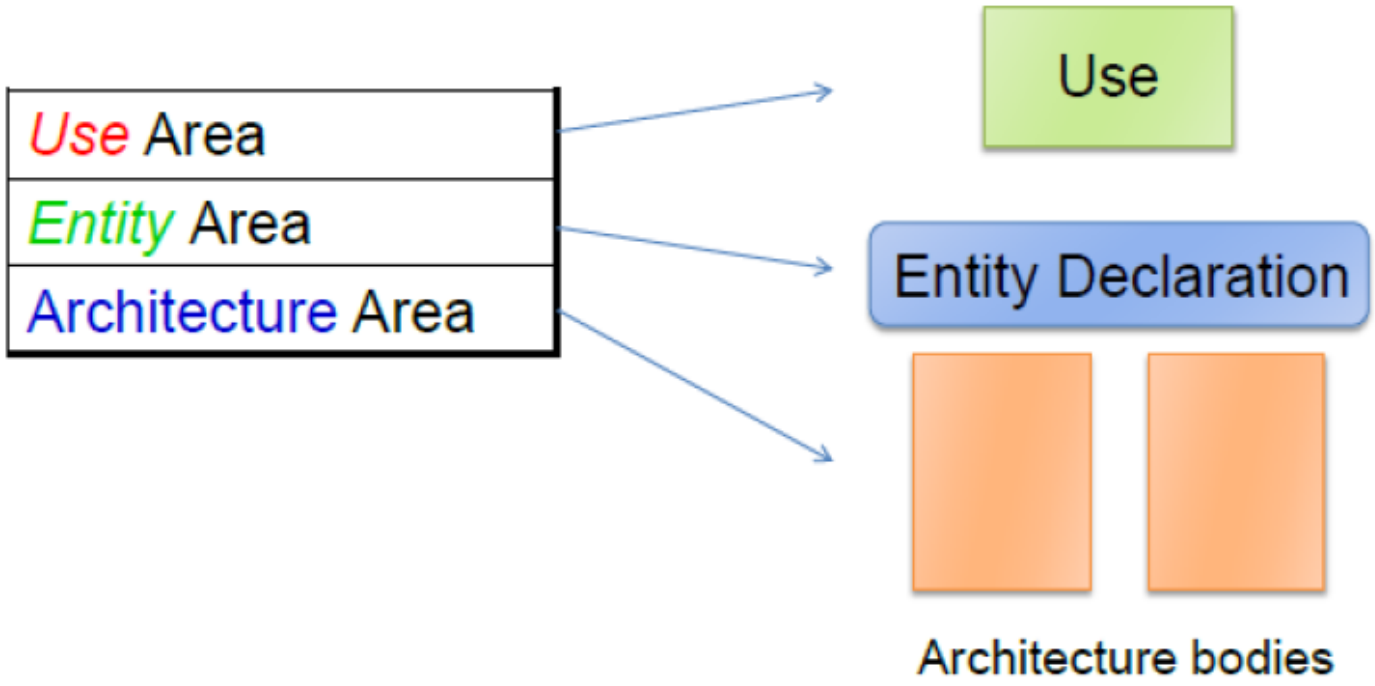
- VHDL is a language that is used to describe the behavior of digital circuit designs

Very High Speed Integrated Circuit
Hardware
Description
Language

- VHDL designs can be simulated and translated into a form suitable for hardware implementation

- Hierarchical use of VHDL designs permits the rapid creation of complex digital circuit designs

Code Structure



Use Area/Library and Packages

- Libraries provide a set of packages, components, and functions that simplify the task of designing hardware
- Packages provide a collection of related data types and subprograms
- The following is an example of the use of the `ieee` library and its `std_logic_1164` package:

```
LIBRARY ieee;

USE ieee.std_logic_1164.ALL;
```

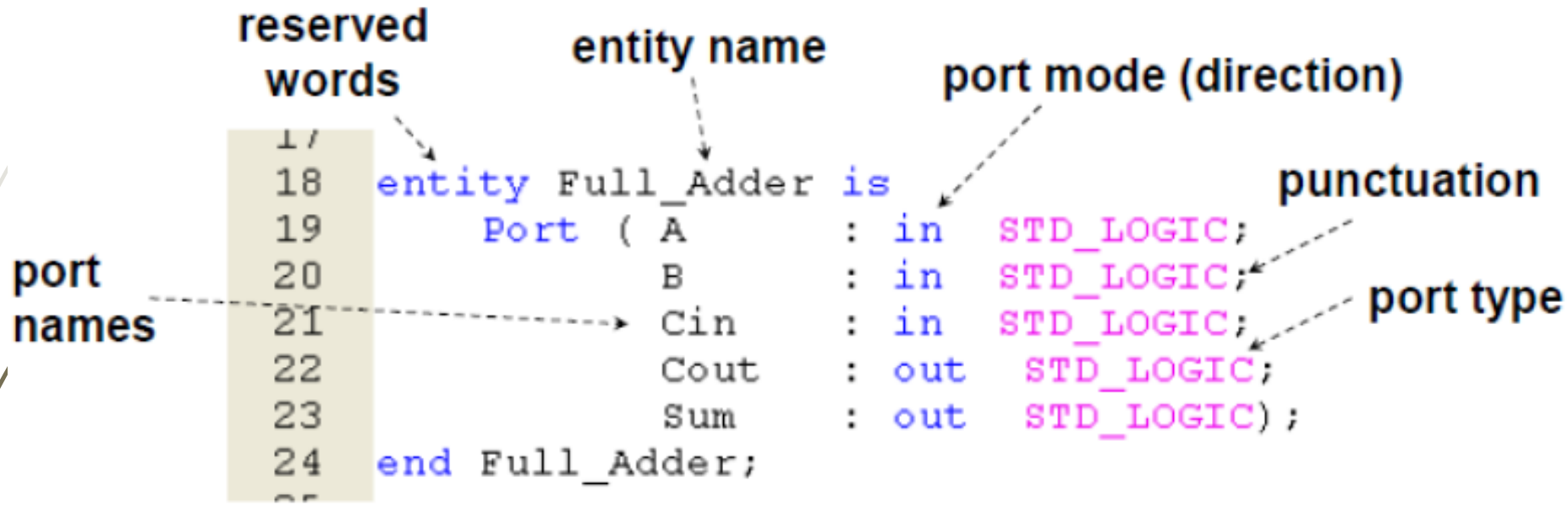
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
```

Entity

- The Entity is used to specify the input and output ports of the circuit. An Entity usually has one or more ports that can be inputs (in), outputs (out), input-outputs (inout), or buffer.
- An Entity may also include a set of generic values that are used to declare properties of the circuit.

```
ENTITY entity_name IS
    GENERIC(
            generic_1_name      : generic_1_type;
            generic_2_name      : generic_2_type;
            generic_n_name      : generic_n_type
    );
    PORT(
            port_1_name   : port_1_dir  port_1_type;
            port_2_name   : port_2_dir  port_2_type;
            port_n_name   : port_n_dir  port_n_type
    );
END entity_name;
```

- Example: Full Adder



Ports

- Port name choices:
  - Consist of letters, digits, and/or underscores
  - Always begin with a letter
  - Case insensitive
- Port direction choices:

`IN`	Input port
`OUT`	Output port
`INOUT`	Bidirectional port
`BUFFER`	Buffered output port

NOTE:

A buffer is an output that can be "read" by the architecture of the entity.

Architectures

- An architecture is a specification of the design's internal implementation
- Multiple architectures can be created for a particular entity
- For example, you might wish to create several architectures for a particular entity with each architecture optimized with respect to a design goal:
  - Performance
  - Area
  - Power Consumption
  - Ease of Simulation

Architecture Bodies

- Architecture declarations are specified as follows:

```
ARCHITECTURE architecture_name OF entity_name IS
BEGIN
    -- Insert VHDL statements to assign outputs to
    -- each of the output signals defined in the
    -- entity declaration.
END architecture_name;
```

```
ARCHITECTURE synthesis1 OF andgate IS
BEGIN
   c <= a AND b;
END synthesis1;
```

1. **Arithmetic Operators:**
   - `+` Addition
   - `-` Subtraction
   - `*` Multiplication
   - `/` Division
   - `mod` Modulus
   - `rem` Remainder
2. **Relational Operators:**
   - `=` Equal to
   - `/=` Not equal to
   - `<` Less than
   - `<=` Less than or equal to
   - `>` Greater than
   - `>=` Greater than or equa
3. **Logical Operators:**
   - `and` Logical AND
   - `or` Logical OR
   - `not` Logical NOT
   - `xor` Logical XOR
   - `nand` Logical NAND
   - `nor` Logical NOR
4. **Shift Operators:**
   - `sll` Shift left logical
   - `srl` Shift right logical
   - `sla` Shift left arithmetic
   - `sra` Shift right arithmetic
   - `rol` Rotate left
   - `ror` Rotate right
5. **Concatenation Operators:**
   - `&` Concatenation operator (for concatenating bits or vectors)
6. **Others:**
   - `=>` Association operator (used in port maps)
   - `:=` Signal assignment operator
   - `<=` Signal assignment operator (for sequential processes)
7. **Conversion Functions:**
   - `to_integer`, `to_unsigned`, `to_signed`: Converts between different integer types and vectors.

1. **Standard Logic (`std_logic`):**
   - Represents a single digital value, which can take one of nine states: 'U' (uninitialized), 'X' (unknown), '0' (logic low), '1' (logic high), 'Z' (high-impedance), 'W' (weak unknown), 'L' (weak logic low), 'H' (weak logic high), '-' (don't care).
2. **Standard Logic Vector (`std_logic_vector`) and Bit Vector (`bit_vector`):**
   - `std_logic_vector`: Represents an array of standard logic values.
   - `bit_vector`: Similar to `std_logic_vector` but can only hold '0' or '1' values.
3. **Integer:**
   - Represents integer values.
4. **Boolean:**
   - Represents Boolean values (`true` or `false`).
5. **Enumeration Types:**
   - User-defined types where a variable can only take one value from a predefined set of values.
6. **Arrays:**
   - Allows the declaration of one-dimensional and multi-dimensional arrays of any data type.
7. **Record Types:**
   - User-defined composite data types that can hold multiple elements of different data types.
8. **Physical Types:**
   - Represents physical quantities like time, length, etc., with associated units.
9. **File Types:**
   - Represents file objects used for file I/O operations.

VHDL Modeling Style

- Architecture body defines the internal details of an entity using any of the following modelling styles:
  - Dataflow Modelling:

    through concurrent assignment statements
  - Behavioural Modelling:

    through sequential assignment statements
  - Structural Modelling:

    through interconnecting components
  - Mixed Modelling: A mixture of structural, dataflow and behavioural modelling

VHDL Modeling Style: Dataflow Example

Full Adder Equations:

- Sum = A ⊕ B ⊕ Cin
- Cout = A · B + Cin · A + Cin · B

```
26  architecture dataflow of Full_Adder is
27
28  begin
29
30  Sum <= A xor B xor Cin;
31
32  Cout <= (A and B) or (A and Cin) or (B and Cin);
33
34  end dataflow;
```

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity or_gate is
    Port (
        A : in  STD_LOGIC;
        B : in  STD_LOGIC;
        Y : out STD_LOGIC
    );
end or_gate;

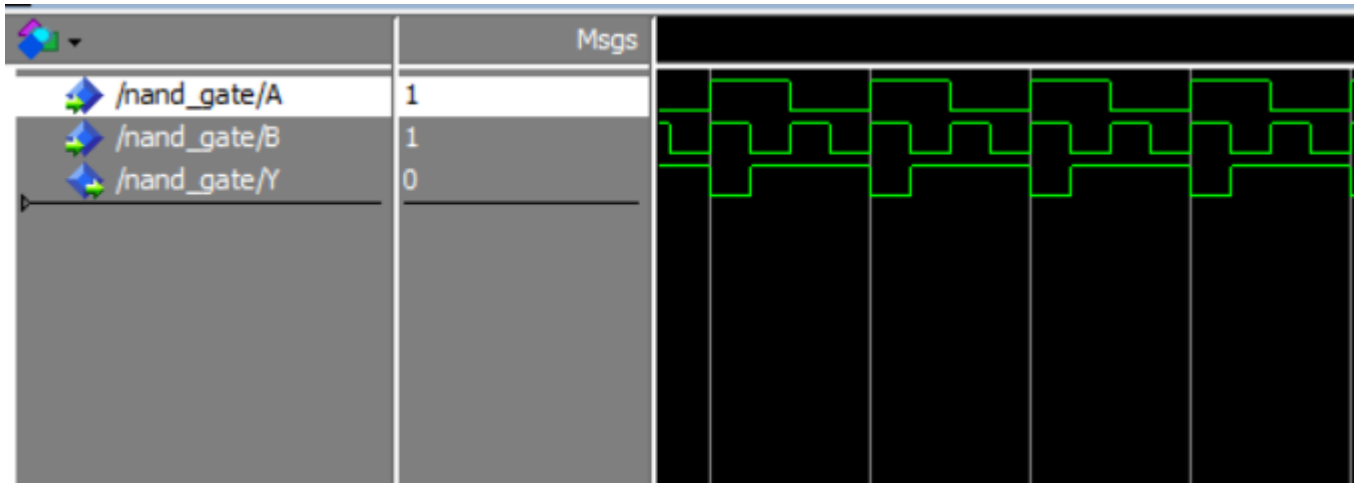
architecture Dataflow of or_gate is
begin
    -- Dataflow description: directly assign logic equation
    Y <= A or B;
end Dataflow;
```

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity nand_gate is -- Starting Entity
    Port (
        A : in  STD_LOGIC;
        B : in  STD_LOGIC;-- Port declaration
        Y : out STD_LOGIC
    );
end nand_gate; -- Ending entity

architecture Dataflow of nand_gate is --Starting Architecture
begin
    -- Dataflow description: directly assign logic equation
    Y <= A nand B;
end Dataflow; -- Ending architecture

```



VHDL Modeling Style: Behavioral

- The kind of description realizing sequential logic, i.e. flip-flops.
- Used when sequential execution is needed.

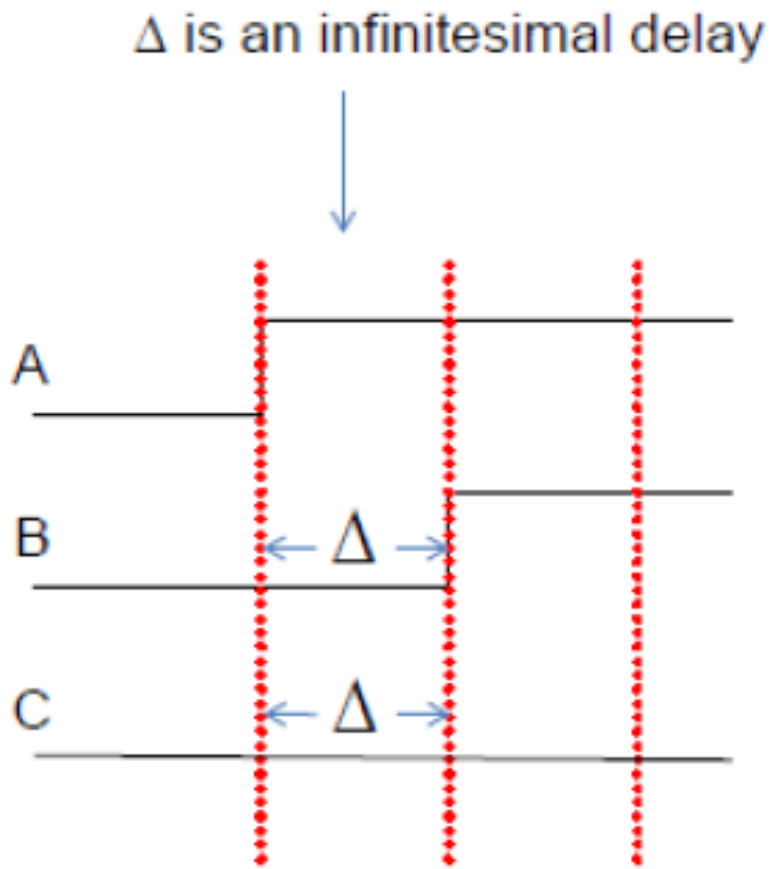
```
[label:] process (sensitivity-list)
...
begin
...
end process [label];
```

- The process executes whenever any signal in the sensitivity list changes.

VHDL Modeling Style: Behavioral example

- Behavioral

```
23  architecture Behavioral of sequential is
24  begin
25      process (A)
26      begin
27          B <= A;
28          C <= B;
29      end process;
30  end Behavioral;
```

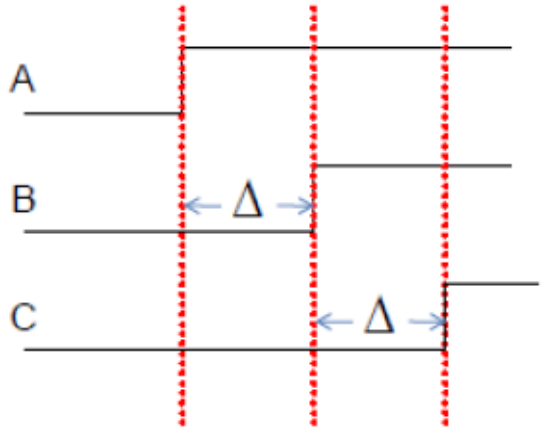


VHDL Modeling Style: Behavioral example

- Behavioral

```
23  architecture Behavioral of sequential is
24  begin
25      process (A,B)
26      begin
27          B <= A;
28          C <= B;
29      end process;
30  end Behavioral;
```

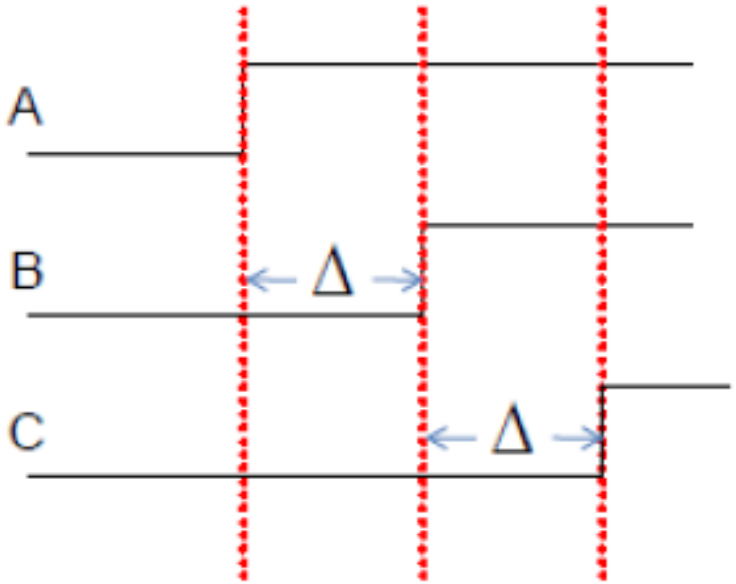
One more sensitive signal



VHDL Modeling Style: Behavioral example

- Dataflow

```
23  architecture dataflow of concurrent is
24  begin
25      B <= A;
26      C <= B;
27  end dataflow;
```



```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

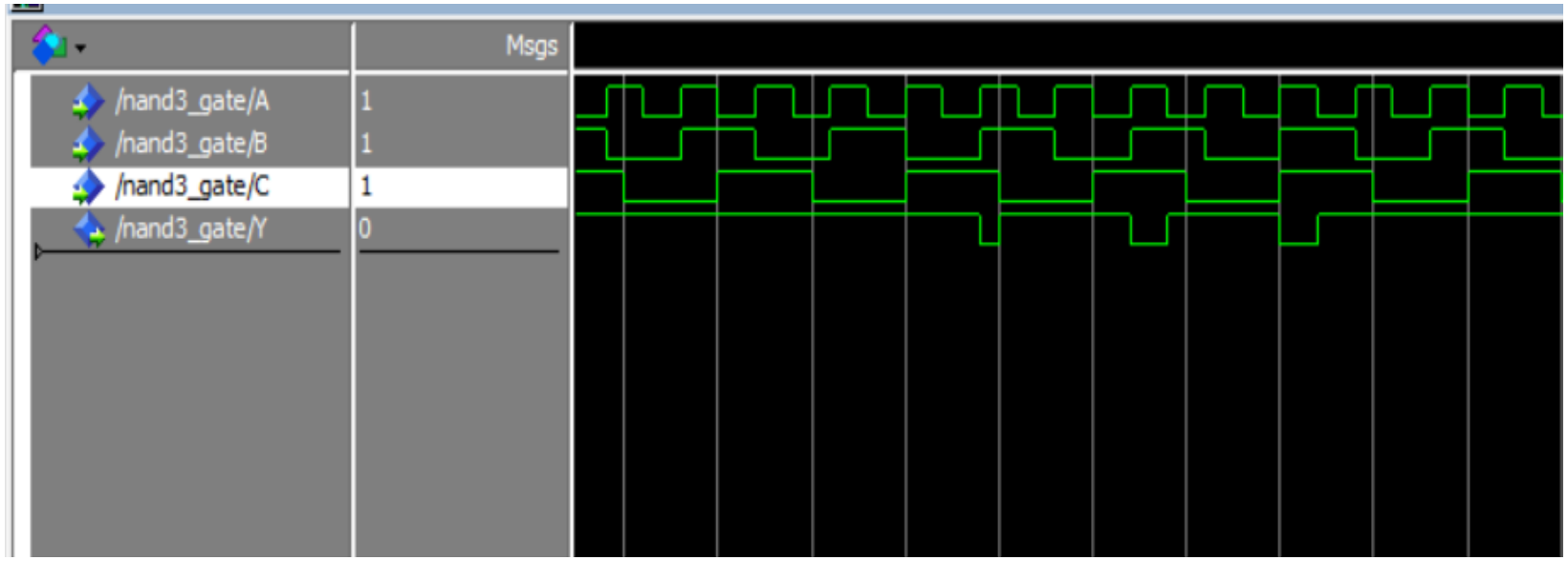
entity or_gate is
    Port (
        A : in  STD_LOGIC;
        B : in  STD_LOGIC;
        Y : out STD_LOGIC
    );
end or_gate;

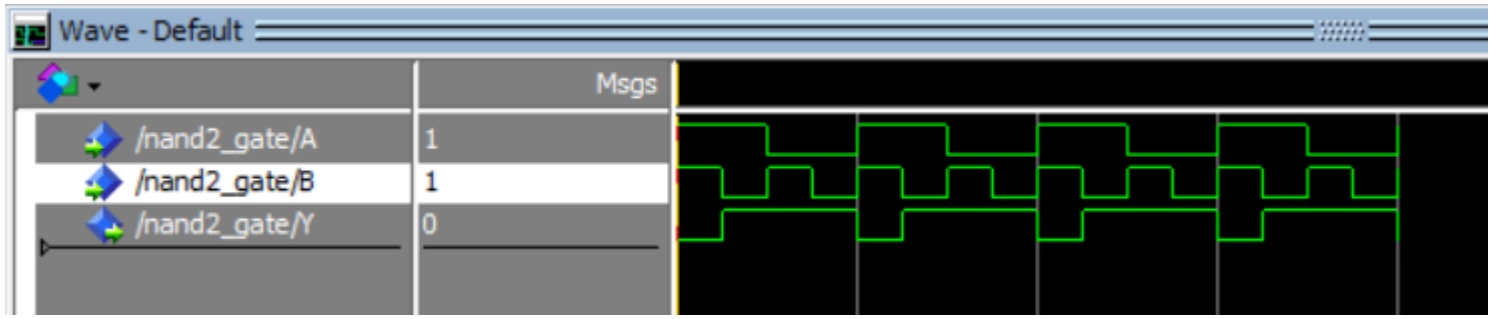
architecture Behavioral of or_gate is
begin
    process(A, B)
    begin
        if (A = '1' or B = '1') then
            Y <= '1';
        else
            Y <= '0';
        end if;
    end process;
end Behavioral;
```

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity NAND3_Gate is
    Port (
        A : in  STD_LOGIC;
        B : in  STD_LOGIC;
        C : in  STD_LOGIC;
        Y : out STD_LOGIC
    );
end NAND3_Gate;

architecture Behavioral of NAND3_Gate is
begin
    process(A, B, C)
    begin
        if (A = '1' and B = '1' and C = '1') then
            Y <= '0';
        else
            Y <= '1';
        end if;
    end process;
end Behavioral;
```



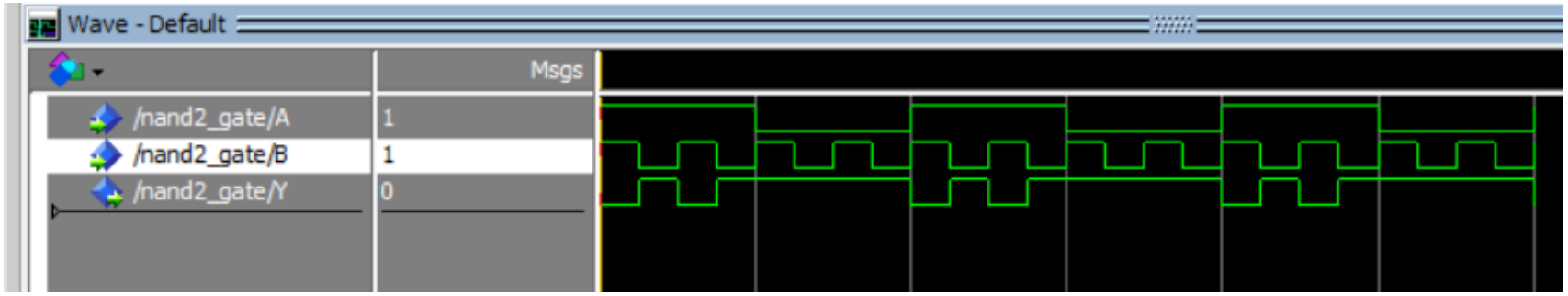



```
 2  library IEEE;
 3  use IEEE.STD_LOGIC_1164.ALL;
 4
 5  entity NAND2_Gate is
 6      Port (
 7          A : in  STD_LOGIC;
 8          B : in  STD_LOGIC;
 9          Y : out STD_LOGIC
10      );
11  end NAND2_Gate;
12
13  architecture Behavioral of NAND2_Gate is
14  begin
15      -- Direct behavioral description using if-else in a conditional signal assignment
16      Y <= '0' when (A = '1' and B = '1') else
17           '1';
18  end Behavioral;
```

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

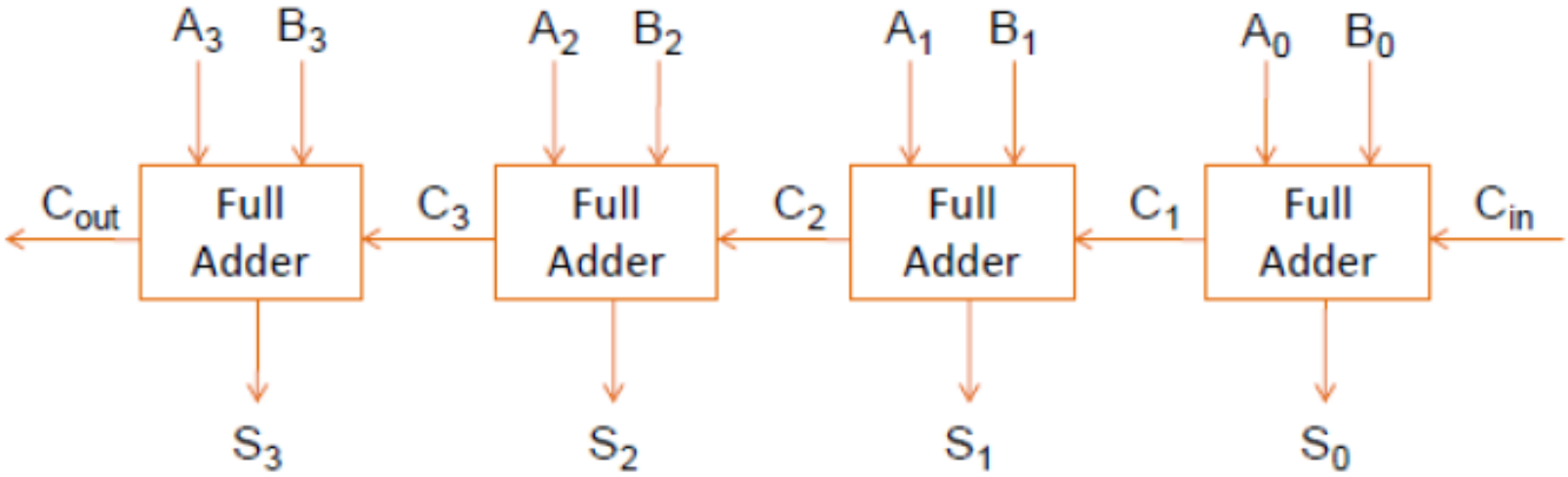
entity NAND2_Gate is
    Port (
        A : in  STD_LOGIC;
        B : in  STD_LOGIC;
        Y : out STD_LOGIC
    );
end NAND2_Gate;

architecture Behavioral of NAND2_Gate is
begin
    process(A, B)
    begin
        if (A = '0' and B = '0') then
            Y <= '1';   -- NAND(0,0) = 1
        elsif (A = '0' and B = '1') then
            Y <= '1';   -- NAND(0,1) = 1
        elsif (A = '1' and B = '0') then
            Y <= '1';   -- NAND(1,0) = 1
        else
            Y <= '0';   -- NAND(1,1) = 0
        end if;
    end process;
end Behavioral;
```



VHDL Modeling Style: Structural Example

- A 4-bit adder is composed of 4 full adders.
- VHDL: interconnect 4 full adders into a 4-bit adder
- Using structural description style
- A good example of hierarchy design



VHDL Modeling Style: Structural Example

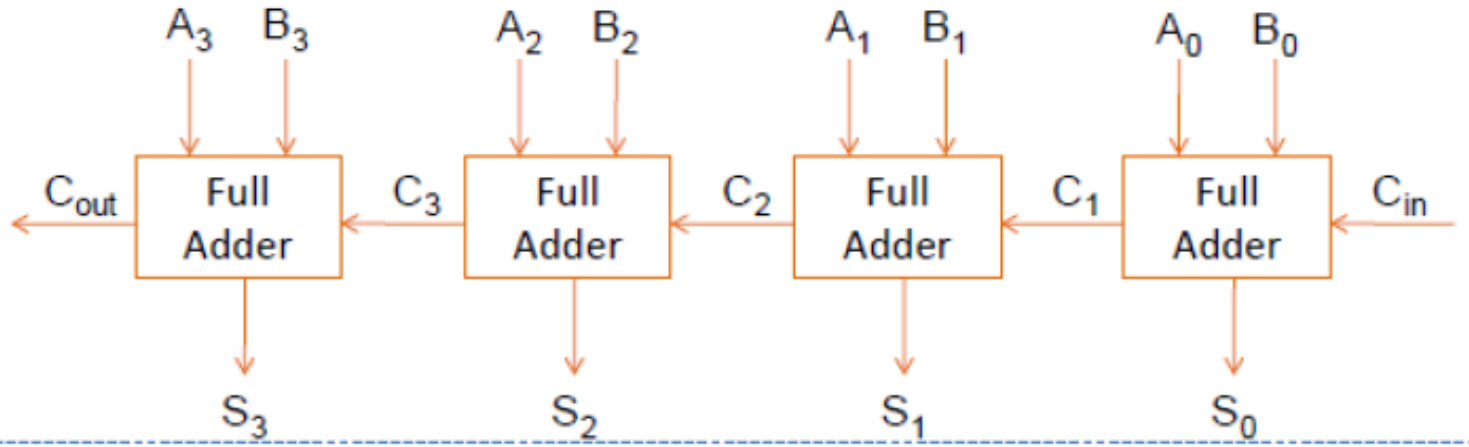
```
component Full_Adder
   port (A, B, Cin : in STD_LOGIC;
         Cout, Sum : out STD_LOGIC);
end component;
```

- Component Instantiation:
  - Named Association

FA1: Full_Adder port map (A=>A_1, B=>B_1, Cin=>C_1, Cout=>C_2, Sum=>S_1);

  - Position association

FA1: Full_Adder port map (A_1, B_1, C_1, C_2, S_1);



VHDL Modeling Style: Structural Example

- Code for a 4-bit adder

  by structurally interconnecting 4 full adders

```
25  architecture structural of Adder4 is
26
27  component Full_Adder
28     port (A, B, Cin : in STD_LOGIC;
29           Cout, Sum : out STD_LOGIC);
30  end component;
31
32  signal C : std_logic_vector (3 downto 1);
33
34  begin
35
36  FA0: Full_Adder port map (A(0),B(0),Cin,C(1),S(0));
37
38  FA1: Full_Adder port map (A(1),B(1),C(1),C(2),S(1));
39
40  FA2: Full_Adder port map (A(2),B(2),C(2),C(3),S(2));
41
42  FA3: Full_Adder port map (A(3),B(3),C(3),Cout,S(3));
43
44  end structural;
45
```

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity AND2 is
   Port (
      A : in  STD_LOGIC;
      B : in  STD_LOGIC;
      Y : out STD_LOGIC
   );
end AND2;

architecture Behavioral of AND2 is
begin
   Y <= A and B;
end Behavioral;

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity NOT1 is
   Port (
      A : in  STD_LOGIC;
      Y : out STD_LOGIC
   );
end NOT1;

architecture Behavioral of NOT1 is
begin
   Y <= not A;
end Behavioral;

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity NAND2_Structural is
   Port (
      A : in  STD_LOGIC;
      B : in  STD_LOGIC;
      Y : out STD_LOGIC
   );
end NAND2_Structural;
```




```vhdl
architecture Structural of NAND2_Structural is
   -- Internal signal
   signal and_out : STD_LOGIC;

   -- Declare components
   component AND2 is
      Port (
         A : in  STD_LOGIC;
         B : in  STD_LOGIC;
         Y : out STD_LOGIC
      );
   end component;

   component NOT1 is
      Port (
         A : in  STD_LOGIC;
         Y : out STD_LOGIC
      );
   end component;

begin
   -- Instantiate components
   U1: AND2 port map (A => A, B => B, Y => and_out);
   U2: NOT1 port map (A => and_out, Y => Y);

end Structural;
```

```vhdl
begin
   -- Instantiate components
   U1: AND2 port map ( A, B,and_out);
   U2: NOT1 port map ( and_out, Y);
```

VHDL Process

- Two types
  - Process with a sensitivity list
  - Process with wait statements
  - A process cannot have both a sensitivity list and wait statements, but must have one of them.
- Statements within a process are called sequential statements as they execute in sequence, e.g. IF statements.
- Statements outside a process are called concurrent statements as they execute concurrently.
- Signal assignment statement (C <= A and B) can be either concurrent or sequential.

```
process (A, B)
begin
   Y <= A and B;
end process;
```

```
wait on A, B;

wait until clk = '1';

wait until rising_edge(clk);
```

VHDL Processes Statements

- The keywords used for conditional assignments and selected assignments differ from those used within a process:

Outside Processes	Inside Processes
WHEN..ELSE	IF..ELSIF..ELSE..END IF
WITH..SELECT..WHEN	CASE..WHEN..END CASE

- A selected assignment outside a process is functionally equivalent to a case statement within a process
- Processes can be used for combinational logic but most often, processes encapsulate sequential logic

VHDL Processes Statements…Continue

```
SIGNAL reset, clock, d, q          :std_logic;

PROCESS (reset, clock)
-- reset and clock are in the sensitivity list to
-- indicate that they are important inputs to the process
BEGIN
    -- IF keyword is only valid in a process
    IF (reset = '0') THEN
          q <= 0;
    ELSIF (clock'EVENT AND clock = '1') THEN
          q <= d;
    END IF;
END PROCESS;
```

The EVENT attribute is true if an edge has been detected on the corresponding signal.

NOTE:

This implements a D flip-flop with an asynchronous active-low reset signal.

VHDL Processes Statements…Continue

```
SIGNAL a, b, c, d          :std_logic;

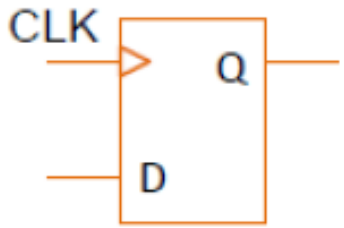
PROCESS (a, b, d)
-- a, b, and d are in the sensitivity list to indicate that
-- the outputs of the process are sensitive to changes in them
BEGIN
    -- CASE keyword is only valid in a process
    CASE d IS
          WHEN '0' =>
                    c <= a AND b;
          WHEN OTHERS =>
                    c <= '1';
    END CASE;
END PROCESS;
```

NOTE:

This implements a combinational circuit.

Flip-Flops Modeling—A D flip-flop

```
process(CLK)
begin
      if  CLK'event and CLK = '1'
          then Q<= D;
      end if;
end process;
```



rising edge of CLK

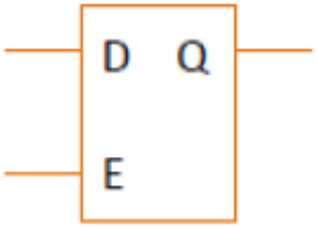
'event – predefined attribute for any signal

CLK'event return's TRUE whenever CLK changes.

If CLK = '1' is also TRUE, it means the change was from '0' to '1' – a rising edge.

Flip-Flops Modeling—A transparent Latch

```
process(E, D)
begin
      if  E = '1'
          then Q<= D;
      end if;
end process;
```



- When E = '1', D passes to Q.
  When E = '0', Q does not change.

If Statements

```
process
begin
        ……
        if  condition then sequential statements;
            [elsif condition then sequential statements;]
                ……
            [elsif condition then sequential statements;]
            [else sequential statements;]
        end if;
        ……
end process;
```

- If statements are sequential statements:

  They have to be used within a process, and cannot be used outside a process.

Process using Wait Statement

```
process
begin
        sequential statements;
        wait statement;
        sequential statements;
        wait statement;
        ……
end process;
```

- must have no sensitivity list following word “process”

Wait Statement—wait on

- wait on sensitivity list
  - waits until one of the signals on the sensitivity list changes
  - for example, wait on A, B, C;
  - equivalent processes:

```
process(A, B, C, D)
begin
    C <= A and B after 5 ns;
    E <= C or D after 5 ns;
end process;
```

```
process
begin
    C <= A and B after 5 ns;
    E <= C or D after 5 ns;
    wait on A, B, C, D;
end process;
```

Wait Statement—wait for & wait until

- wait for time expression
  - waits until the time has lapsed
  - for example, wait for 5 ns;
  - not synthesizable, but very useful in writing VHDL simulation code

- wait until Boolean expression
  - waits until the expression evaluates to TRUE
  - for example, wait until A = B;

VHDL Programming—Some Basic Syntax for Coding

- VHDL is not case sensitive
- Semicolon ";" at the end of each statement
- Comments appear after a double dash "--"
- Signal assignment operator "<="

  exactly the same as "less than or equal to"
- VHDL identifiers (signal or variable names) may contain letters, numbers and underscores ('_'), must start with a letter, and cannot end with an '_' or contain successive '_'.
- A signal in VHDL usually corresponds to a signal in a physical system

VHDL Signal Assignment

- General Signal assignment statement:

  signal_name <= expression [after delay]

  […] means "…" is optional.

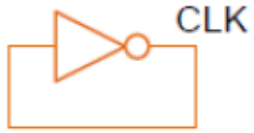


Example: C <= A and B [after 5 ns];

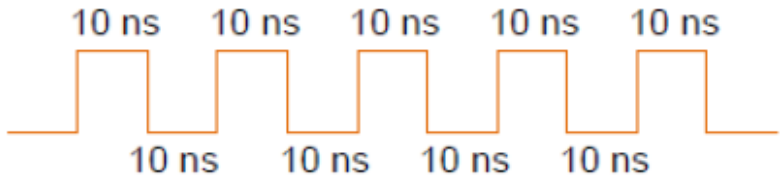
statement example	propagation delay of statement	time of C resulted after A or B changes in reality
C <= A and B;	Δ (An infinitesimal delay)	the actual propagation delay of A and B
C <= A and B after 5 ns;	5 ns	5 ns

VHDL Signal Assignment…Continue

- Clock Generation



  - CLK <= not CLK after 10 ns
  - clk <= NOT clk After 10 NS



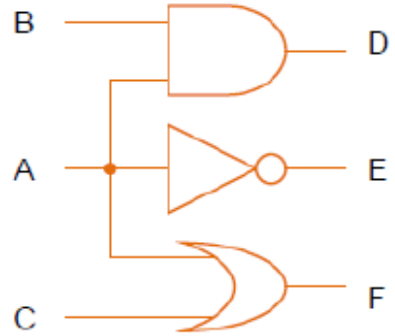
- Clock cycle = 20 ns, Frequency = 50 MHz

VHDL – Concurrent Assignment

D <= A and B after 2 ns;

E <= not A after 3 ns;

F <= A or C after 4 ns;



Signals vs. Variables:

- Variables can only be used inside processes, signals can be used inside or outside processes.
- Any variable that is created in one process cannot be used in another process, signals can be used in multiple processes *though they can only be assigned in a single process*.
- Variables need to be defined after the keyword *process* but before the keyword *begin*. Signals are defined in the architecture before the *begin* statement.
- Variables are assigned using the **:=** assignment symbol. Signals are assigned using the **<=** assignment symbol.
- Variables that are assigned immediately take the value of the assignment. Signals depend on if it's combinational or sequential code to know when the signal takes the value of the assignment.